

10

Preserving data

This chapter demonstrates how to preserve data using cookies and PHP sessions.

Submitting cookie data

Setting cookies

Getting cookies

Viewing cookie data

Submitting session data

Setting sessions

Getting sessions

Viewing session data

Summary

Submitting cookie data



Data entered by a user is frequently required across multiple pages of a website. User data can conveniently be stored in small “cookie” files on the client machine, so the user need not tediously enter the same data repeatedly. The PHP **setcookie()** function makes it easy to create a cookie by specifying three arguments:

setcookie(*name* , *value* , *expiry-time*) ;

When an HTML form is submitted to the server using the **POST** method, its field data is automatically assigned to the special PHP **\$_POST** global array variable. A script can check for the presence of individual submission fields using a built-in PHP **isset()** function to seek an element of a specified HTML field name. When this confirms the field is indeed present, its name and value can usefully be stored in a cookie. This might be used to store username and password details for use across a website:

- 1 Create an HTML document and add this form to its body section, specifying submission method and the name of a PHP submission-handler script

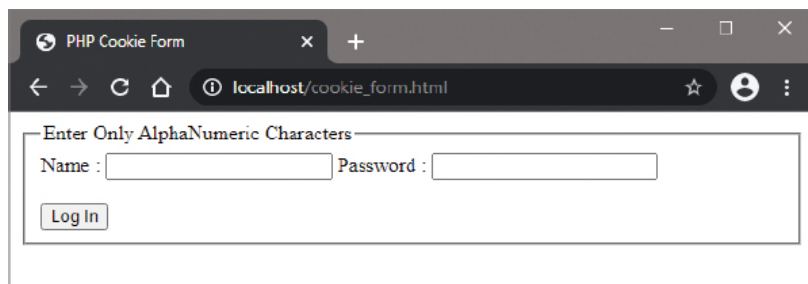
```
<form name = "entry" method = "POST"
        action = "cookie_set.php">

<fieldset>
<legend>Enter Only AlphaNumeric Characters</legend>
Name : <input type = "text" name = "user" >
Password : <input type = "password" name = "pass" >
<br><br><input type = "submit" value = "Log In" >
</fieldset>
</form>
```



cookie_form.html

2 Save the document in your web server's **/htdocs** directory as **cookie_form.html** then open the page via HTTP to see the form



The screenshot shows a web browser window with the title 'PHP Cookie Form'. The address bar displays 'localhost/cookie_form.html'. The form contains a heading 'Enter Only AlphaNumeric Characters', two input fields labeled 'Name :' and 'Password :', and a 'Log In' button.



The **setcookie()** function can also accept arguments to specify visible path, domain, https security, and visibility for http access.



Storing passwords in cookies is not the most secure method but is used here merely to demonstrate how to use cookies to store data.

Setting cookies

Upon receiving a form submission, a PHP script can check for the presence of fields and might perform validation of their associated values. For example, the **ctype_alnum()** function could be used to simply confirm that the values only contain alphanumeric data.

Once validated, the **setcookie()** function can store the submitted fields' names and values. Additionally, a hash conversion could be made of password values using the **md5()** function. On successful completion, the script can then relocate the browser to another page using the **header()** function to specify that page's URL:

1 Begin a PHP script with a function to handle failed validation attempts

```
<?php
function reject( $entry )
{
    echo "Invalid $entry <br>" ;
    echo 'Please <a href="cookie_form.html">Log In</a>' ;
    exit() ;
}
```



cookie_set.php

2 Next, add a conditional test block to set two cookies for valid data, or call the function to handle failed attempts

```
if( isset( $_POST[ 'user' ] ) )
{
    $user= trim( $_POST[ 'user' ] ) ;
    if( !ctype_alnum( $user ) ) { reject( 'User Name' ) ; }

    if( isset( $_POST[ 'pass' ] ) )
    {
        $pass = trim( $_POST[ 'pass' ] ) ;
        if( !ctype_alnum( $pass ) ) { reject( 'Password' ) ; }
```

```

else
{
    setcookie( 'user' , $user , time()+3600 ) ;
    setcookie( 'pass' , md5( $pass ) , time()+3600 ) ;
    header( 'Location: cookie_get.php' ) ;
}
}
}
else { header( 'Location: cookie_form.html' ) ; }
?>

```

3 Save the document in your web server's **/htdocs** directory as **cookie_set.php** then see [here](#) for how to retrieve the stored cookie data



The **ctype_alnum()** function returns true only if its argument contains alphanumeric characters.



Notice how the **time()** function is used here to get the current time, then one hour is added (in seconds) to set the cookie expiration time one hour ahead.



The final **else** statement will relocate to the HTML form page if the script URL is opened directly.

Getting cookies



Once cookies are set, they are automatically assigned to the special PHP `$_COOKIE` global array variable. A script can check for the presence of an individual cookie using the built-in PHP `isset()` function to seek a cookie of a specified name. When this confirms the cookie is indeed present, its value can usefully be assigned to a regular script variable. This might be used to retrieve a stored username for output on a page. Where a sought cookie is absent, the script can offer an alternative to the user:

1 Begin a PHP script with a test to find a cookie and retrieve data for output upon success

```
<?php
if( isset( $_COOKIE[ 'user' ] ) )
{
    $user = $_COOKIE[ 'user' ] ;
    echo "<h1>Welcome $user !</h1><hr>" ;
    echo '<a href="cookie_data.php">View Cookie</a>' ;
}
```



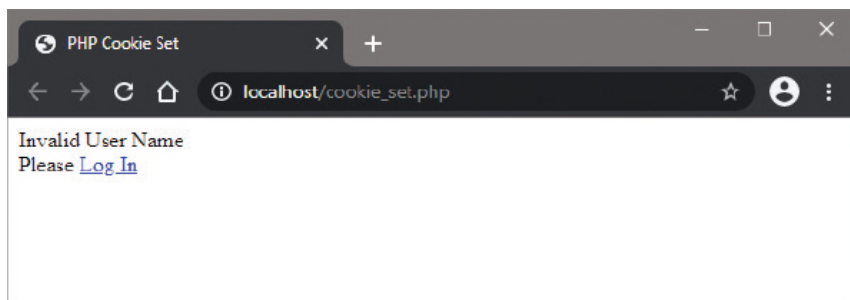
cookie_get.php

2 Next, add alternative output for when the cookie is absent

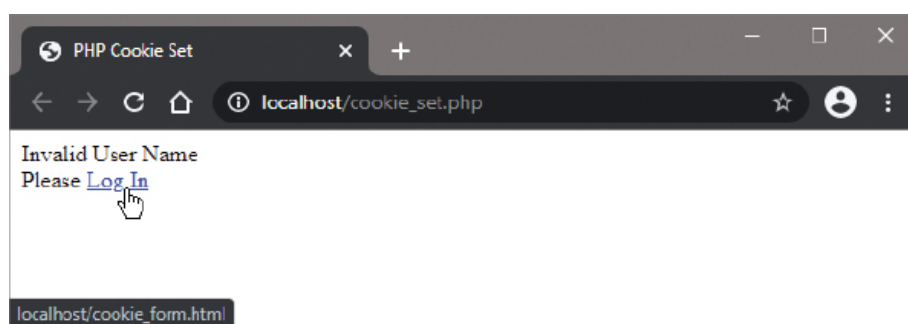
```
else
{
    echo 'Please <a href="cookie_form.html">Log In</a>' ;
}
?>
```

3 Save the document in your web server's `/htdocs` directory as `cookie_get.php`

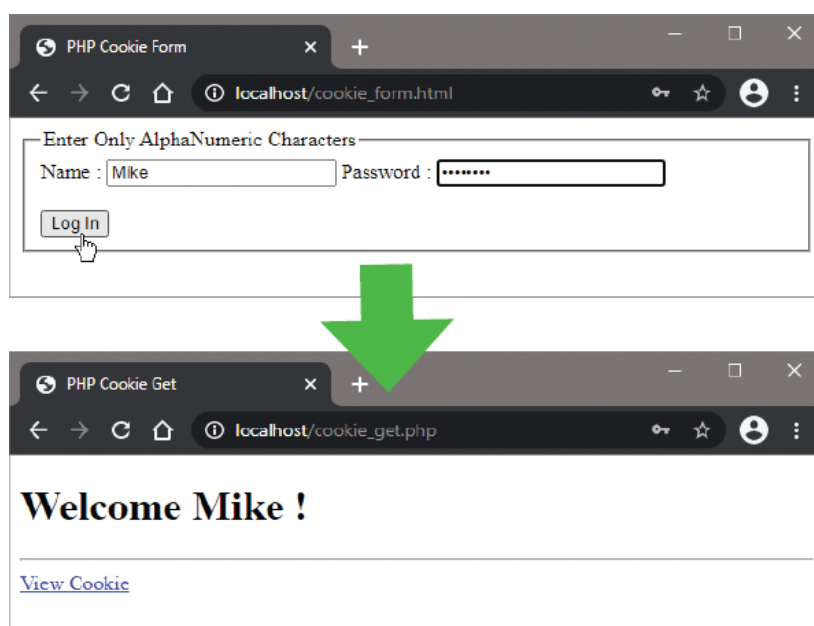
1 Now, open the HTML form **cookie_form.html** (created [here](#)) via HTTP and push the submit button to see the login attempt fail validation, so no cookies are set



5 Click the hyperlink to return to the HTML form and enter a valid username, then push the submit button to see the login attempt fail validation again



6 Return to the HTML form once more and enter a valid username and password to see validation succeed



7 See [here](#) for how to view data stored within cookies, using the **cookie_data.php** target of the hyperlink seen above



You can also return to the HTML form and enter a username or a password that includes non-alphanumeric characters, to see validation fail again.



Validation could be extended. For example, it may also allow underscore characters and might specify a minimum permissible password length.

Viewing cookie data



The special PHP `$_COOKIE` global array variable stores cookie names and values in an associative array of keys and values. Stored content can be viewed by looping through the array to see all names and values, or by using the PHP `var_dump()` function:

- 1 Begin a PHP script with a test to find if any cookies are set and retrieve all stored names and values upon success

```
<?php
if( count( $_COOKIE ) > 0 )
{
    echo '<dl> ' ;
    foreach( $_COOKIE as $key => $value )
    {
        echo "<dt>Key: $key" ;
        echo "<dd>Value: $value" ;
    }
    echo '</dl><hr>' ;
    var_dump( $_COOKIE ) ;
}
```



cookie_data.php

- 2 Next, add alternative output for when cookies are absent

```
else
{
```

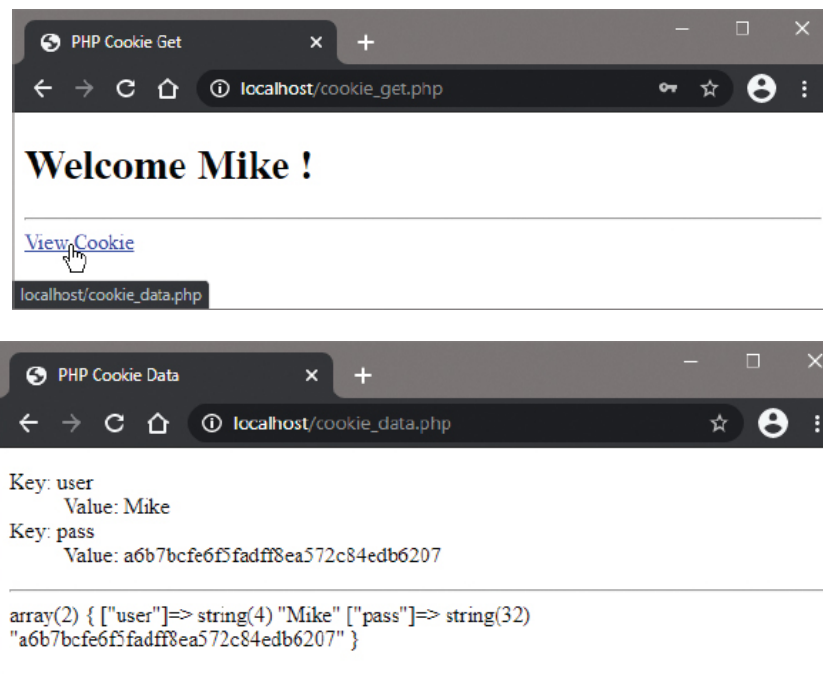
```

echo 'Please <a href="cookie_form.html">Log In</a>' ;
}
?>

```

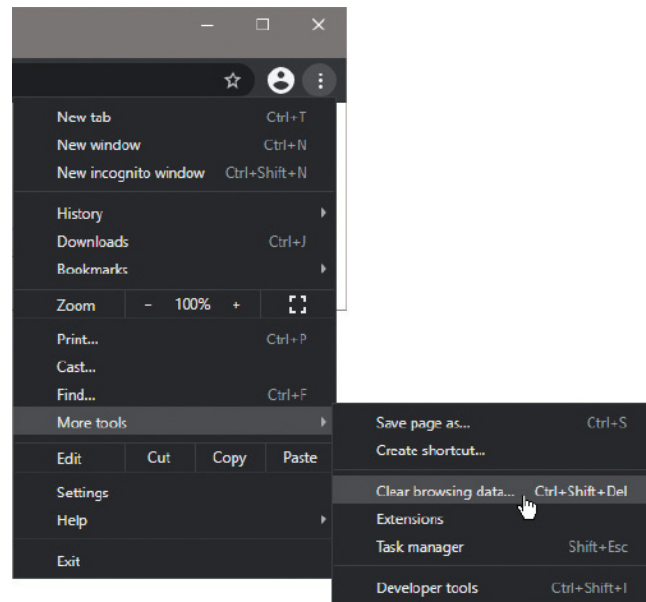
3 Save the document in your web server's **/htdocs** directory as **cookie_data.php**

4 Now, open the HTML form **cookie_form.html** ([here](#)) via HTTP and enter valid login data, then follow the link on **cookie_get.php** ([here](#)) to see cookie data

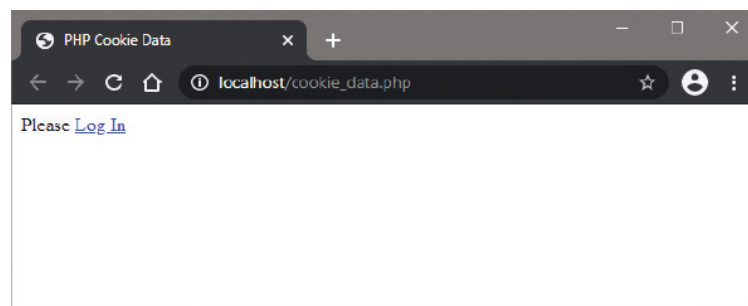


You can call the **setcookie()** function at any time to create a new cookie or overwrite an existing cookie.

5 Use the Settings menu on your web browser to clear existing cookies that are stored on your system



Now, use the Refresh button in your browser to reload this page and see the alternative content – confirming that the cookie data has been removed



The PHP manual describes the removal of a cookie by setting its value to empty and its expiration date to the past. For example, `setcookie($_COOKIE[ 'user' ] , “ , time()-3600 )`. Additionally, the `unset()` function can be used to destroy a specified variable.

Submitting session data



User data can be stored in cookies on the client computer only if the user chooses to allow cookies onto their system. Many users prefer to disallow cookies, however, to avoid tracking – but PHP provides a better alternative. With PHP, user data can be stored on the server using the special `$_SESSION` global array variable. In order for PHP sessions to work, each PHP page must call the built-in `session_start()` function at the top of the page – before any HTML tags! HTML form submissions to the server via the **POST** method assigns field data to the special PHP `$_POST` global array variable, and individual submission fields can be sought using the built-in PHP `isset()` function. When this confirms the field is present, its name and value can be stored in a session variable. This might be used to store username and password details for use across a website, as an alternative to cookie storage:

- 1 Create an HTML document and add this form to its body section, specifying submission method and the name of a PHP submission-handler script

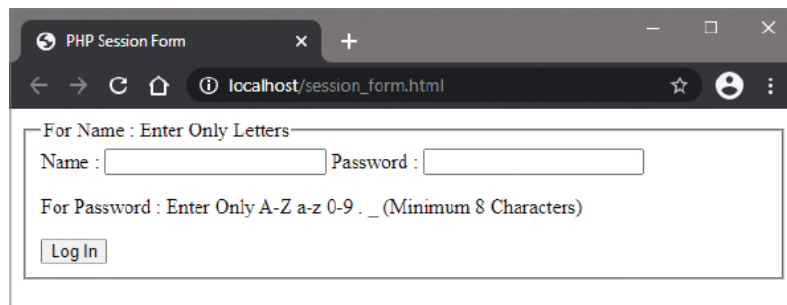
```
<form name = "entry" method = "POST"
        action = "session_set.php">

<fieldset>
<legend>For Name : Enter Only Letters</legend>
Name : <input type="text" name="user" >
Password : <input type="password" name="pass" >
<p>For Password : Enter Only A-Z a-z 0-9 . _ (Minimum 8
Characters)</p> <input type="submit" value="Log In">
</fieldset>
</form>
```



session_form.html

2 Save the document in your web server's **/htdocs** directory as **session_form.html** then open the page via HTTP to see the form



The screenshot shows a web browser window with the title 'PHP Session Form'. The address bar displays 'localhost/session_form.html'. The form contains the following elements:

- A heading: 'For Name : Enter Only Letters'
- Input fields: 'Name : Password : '
- A heading: 'For Password : Enter Only A-Z a-z 0-9 . _ (Minimum 8 Characters)'
- A button: 'Log In'



Session variables will only be accessible to a PHP script if the **session_start()** function has been called at the top of the page.

Setting sessions

Upon receiving a form submission, a PHP script can check for the presence of fields and might perform validation of their associated values. For example, the **ctype_alpha()** function could be used to confirm they contain only letters, and the **preg_match()** function could be used to confirm a value matches a specified pattern. Once validated, the submitted fields' names and values can be assigned to elements of the **\$_SESSION** global array variable. On successful completion, the script can relocate the browser to the next page using the **header()** function to specify that page's URL:

- 1 Add an initial PHP script to make sessions available

```
<?php session_start(); ?>
```



session_set.php

- 2 Begin a PHP script to handle failed validation attempts

```
<?php
function reject( $entry )
{
    echo "Invalid $entry <br>" ;
    echo 'Please <a href="session_form.html">Log In</a>' ;
    exit() ;
}
```

- 3 Add a conditional test block to set two cookies for valid data, or call the function to handle failed attempts

```
if( isset( $_POST[ 'user' ] ) )
{
    $user = trim( $_POST[ 'user' ] ) ;
    if( !ctype_alpha( $user ) ) { reject( 'User Name' ) ; }
    if( isset( $_POST[ 'pass' ] ) )
    {
        $pass = trim( $_POST[ 'pass' ] ) ;
```

```

if( !preg_match( '/^[A-Za-z0-9._]{8,}$/' , $pass ) )
{ reject( 'Password' ) ; }
else
{
    $_SESSION[ 'user' ] = $user ;
    $_SESSION[ 'pass' ] = $pass ;
    header( 'Location: session_get.php' ) ;
}
}
} else { header( 'Location: session_form.html' ) ; }
?>

```

4 Save the document in your web server's **/htdocs** directory as **session_set.php** then see [here](#) for how to retrieve data



The **ctype_alpha()** function returns true only if its argument contains letters.



The **preg_match()** function accepts a regular expression and a string argument. Regular expressions are beyond the remit of this book but you can precisely copy this example to allow the permissible password pattern described on the form. The password is left unencrypted in this example, to demonstrate later that the user password meets the specified requirements.

Getting sessions



Session data is accessible from the special **\$_SESSION** global array variable after a call is made to the **session_start()** function. The PHP **isset()** function can seek an element of a specified name. When this confirms the element is present, its value can usefully be assigned to a regular script variable. This might be used to retrieve a stored username for output on a page. Where a sought element is absent, the script can offer an alternative to the user:

1 Add an initial PHP script to make sessions available

```
<?php session_start() ; ?>
```



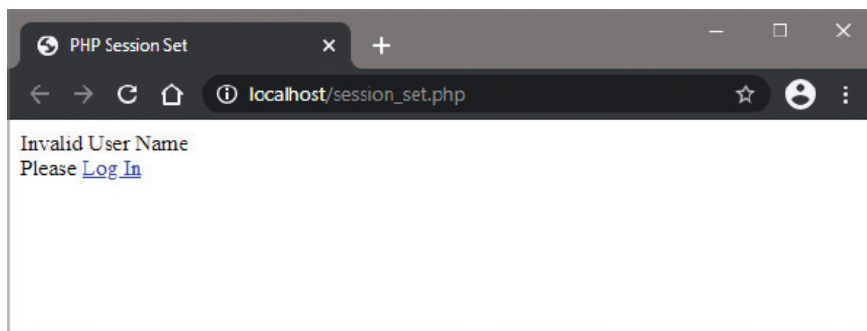
session_get.php

2 Now, begin a PHP script to find a session variable and retrieve data for output upon success, or provide an alternative for when the session variable is absent

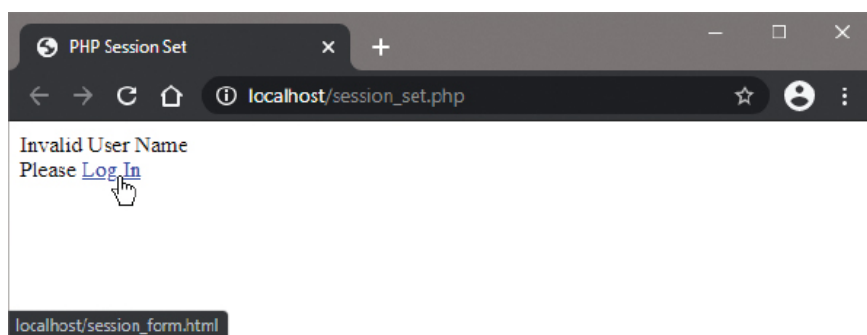
```
<?php
if( isset( $_SESSION[ 'user' ] ) )
{
    $user = $_SESSION[ 'user' ] ;
    echo "<h1>Welcome $user !</h1><hr>" ;
    echo '<a href="session_data.php">View Session</a>' ;
}
else
{ echo 'Please <a href="session_form.html">Log In</a>' ; }
?>
```

3 Save the document in your web server's **/htdocs** directory as **session_get.php**

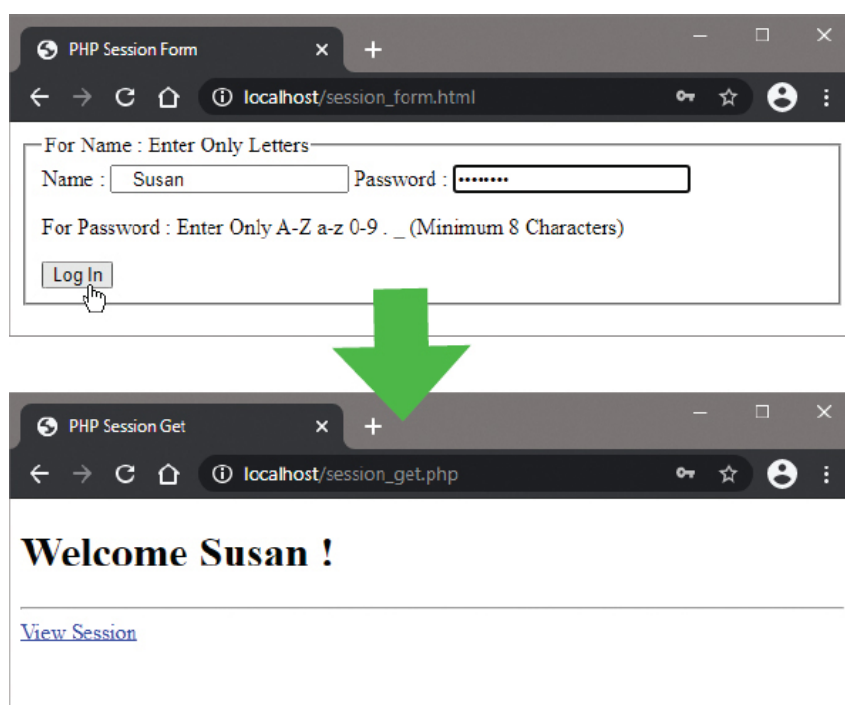
1 Now, open the HTML form **session_form.html** (created [here](#)) via HTTP and push the submit button to see the login attempt fail validation



5 Return to the HTML form and submit a valid username, to see the login attempt fail validation again



6 Return to the HTML form once more and enter a valid username and password to see validation succeed



7 See overleaf how to view data stored in session variables, using the **session_data.php** target of the link seen above



You can also return to the HTML form and enter a password below the minimum length or containing invalid characters to see validation fail again.



Notice that the user in this example has added leading spaces before their name, but that's okay – these will be removed by the **trim()** function used in the **session_set.php** script.

Viewing session data

The special PHP `$_SESSION` global array variable stores names and values in an associative array of keys and values. Stored content can be viewed by looping through the array to see all names and values, or by using the PHP `var_dump()` function. Additionally, each session has a unique session ID number, which can be seen using the PHP `session_id()` function.



Set session variables can be removed by specifying their element name to the PHP `unset()` function, and a session can be completely terminated by calling the `session_destroy()` function:

- 1 Add an initial PHP script to make sessions available

```
<?php session_start(); ?>
```



session_data.php

- 2 Now, begin a PHP script to remove session variables, terminate a session, and confirm termination

```
<?php
function kill_session()
{
    unset( $_SESSION[ 'user' ] );
    unset( $_SESSION[ 'pass' ] );
    session_destroy();
    echo '<hr>Session Destroyed<br>';
    echo 'Session ID : ' . session_id() . '<br>';
    var_dump( $_SESSION );
}
```

}

3 Now, add a test to find if any session variables are set and retrieve all stored names and values

```
if( count( $_SESSION ) > 0 )
{
    echo '<dl> ' ;
    foreach( $_SESSION as $key => $value )
    {
        echo "<dt>Key: $key" ; echo "<dd>Value: $value" ;
    }
    echo '</dl><hr>' ;
    // Statements to be inserted here (Step 4).
}
```

4 Insert statements to display the session ID and array, then call the function to terminate the session

```
echo 'Session ID : '.session_id().'<br>' ;
var_dump( $_SESSION ) ;
kill_session() ;
```



The uniqueness of each session allows variables to exist without conflict.



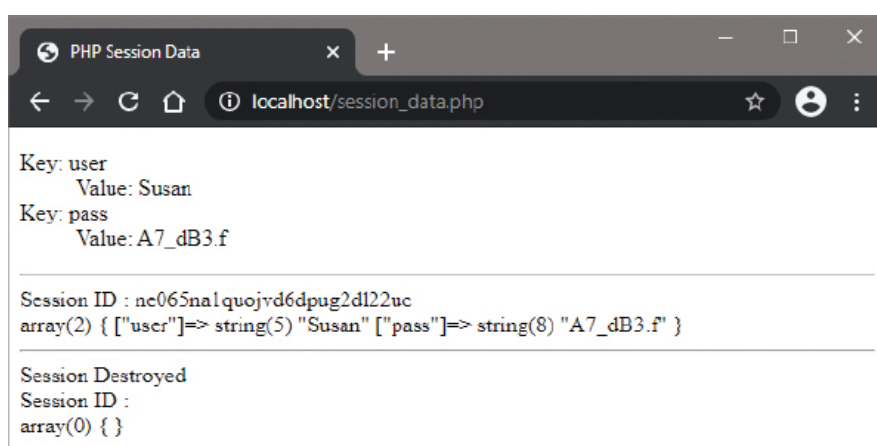
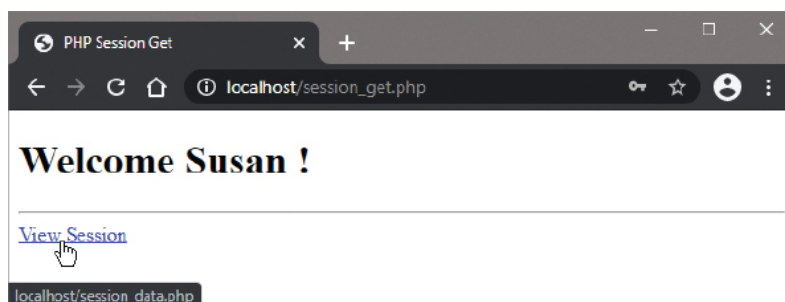
You can assign a new value to a session variable at any time to overwrite an existing stored value.

5 Add alternative output for when a session is absent

```
else
{ echo 'Please <a href="session_form.html">Log In</a>' ; }
?>
```

6 Save the document in your web server's **/htdocs** directory as **session_data.php**

7 Now, open the HTML form **session_form.html** ([here](#)) via HTTP and enter valid login data, then follow the link on **session_get.php** ([here](#)) to see session data



8 Use the  Refresh button in your browser to reload this page to see the alternative “Please Log In” content



Notice that the password adheres to the permissible pattern.

Summary

- The PHP **set_cookie()** function can be used to store user data on the client machine for use across the pages of a website.
- There should be at least three arguments supplied to the **set_cookie()** function to specify name, value and expiry time.
- HTML form submissions via the **POST** method automatically assign field data to the PHP **\$_POST** global array variable.
- The **isset()** function can seek an element of a specified HTML field name within the **\$_POST** global array.
- The **ctype_alnum()** function can be used to confirm a value only contains alphanumeric data for validation.
- A hash conversion can be made of a password value using the **md5()** function for encryption.
- The PHP **header()** function can specify a Location:URL to relocate the browser.
- Once cookies are set, they are automatically assigned to the special PHP **\$_COOKIE** global array variable.
- User data can be stored on the server in the PHP **\$_SESSION** global array variable as an alternative to cookie storage.
- To use sessions, each PHP page must call the **session_start()** function at the top of the page before any HTML tags.
- HTML field data submitted to the **\$_POST** global array can be assigned to a **\$_SESSION** global variable array element.
- The **ctype_alpha()** function can be used to confirm a value only contains alphabetic letter data for validation.
- The **preg_match()** function can be used to confirm a value matches a specified pattern for validation.
- Each session has a unique session ID number, which can be seen using the PHP **session_id()** function.
- Session variables can be removed using the **unset()** function, and a session can be terminated using the **session_destroy()** function.

- Stored content can be viewed using the **var_dump()** function and by looping through the **\$_COOKIE** or **\$_SESSION** arrays.